

Herança em Java - Parte 1

Universidade Tiradentes
Prof. Msc. Felipe Lima

Repetição de Código

Conceito

- Tomemos como exemplo a classe Funcionario, que representa o funcionário de um banco:

```
public class Funcionario {  
    private String nome;  
    private String cpf;  
    private double salario;  
  
    public String getNome() {  
        return nome;  
    }  
  
    public void setNome(String nomeFuncionario) {  
        nome = nomeFuncionario;  
    }  
  
    public String getCpf() {  
        return cpf;  
    }  
}
```

Repetição de Código

Conceito

- Além de um funcionário comum, há também outros cargos, como os gerentes
 - Os gerentes guardam a mesma informação que um funcionário comum, mas possuem outras informações, além de ter funcionalidades um pouco diferentes
 - Vamos supor que, nesse banco, o gerente possui também uma senha numérica que permite o acesso ao sistema interno do banco

Repetição de Código

Conceito

- Classe Gerente

```
public class Gerente {  
    private String nome;  
    private String cpf;  
    private double salario;  
  
    private int senha;  
  
    public boolean autentica(int testarSenha) {  
        if (testarSenha == senha) {  
            System.out.println("Acesso Permitido!");  
            return true;  
        } else {  
            System.out.println("Acesso Negado!");  
        }  
        return false;  
    }  
  
    public String getNome() {  
        return nome;  
    }  
}
```

Repetição de Código

Conceito

- Em vez de criar duas classes diferentes, uma para funcionário e outra para gerente, poderíamos ter deixado a classe Funcionario mais genérica, mantendo nela a senha de acesso
 - Caso o funcionário não fosse um gerente, deixaríamos este atributo vazio (não atribuiríamos valor a ele)
- Mas e em relação aos métodos?
 - A classe Gerente tem o método autentica, que não faz sentido ser acionado em um funcionário que não é gerente

Repetição de Código

Conceito

- Se tivéssemos um outro tipo de funcionário, que tem características diferentes do funcionário comum, precisaríamos criar uma outra classe, e copiar o código novamente!
 - Ou ainda, se precisássemos adicionar uma nova informação (ex: data de nascimento) para todos os funcionários?
 - Todas as classes teriam que ser alteradas para receber essa informação
- **SOLUÇÃO: Centralizar as informações principais do funcionário em um único lugar!**

Herança

Conceito

- Existe uma maneira, em Java, de relacionarmos uma classe de tal maneira que uma delas herde tudo que a outra tem
 - Isso é uma relação de classe mãe e classe filha
- No nosso caso, gostaríamos de fazer com que o **Gerente** tivesse tudo que um **Funcionario** tem:
 - Gostaríamos que ela fosse uma extensão de Funcionario
- **Fazemos isso através da palavra extends**

Herança

Conceito

- Classe Gerente

```
public class Gerente extends Funcionario {  
  
    private int senha;  
  
    public boolean autentica(int testarSenha) {  
        if (testarSenha == senha) {  
            System.out.println("Acesso Permitido!");  
            return true;  
        } else {  
            System.out.println("Acesso Negado!");  
            return false;  
        }  
    }  
}
```


Herança

Conceito

- Todo momento que criarmos um objeto do tipo Gerente, este objeto possuirá também os atributos definidos na classe Funcionario, pois agora **um Gerente É UM Funcionario**

Herança

Conceito

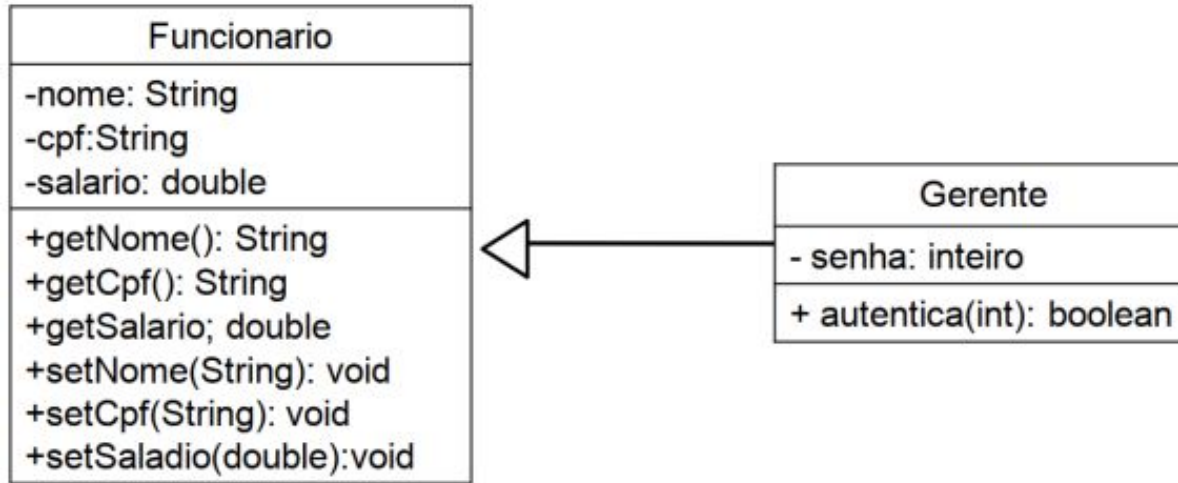
- Termos utilizados

Classes que fornecem Herança	Classes que herdam de outras
Superclasse	Subclasse
Pai	Filha
Tipo	Subtipo

Herança

Conceito

- Notação UML



Herança

Conceito

- Exemplo de teste da classe

```
public class TestarHeranca {  
    public static void main(String[] args) {  
        Gerente gerenteBanco = new Gerente();  
        gerenteBanco.setNome("João da Silva");  
        gerenteBanco.setCpf("123456789101");  
        gerenteBanco.setSenha(123456);  
    }  
}
```

Relacionamento “É UM”

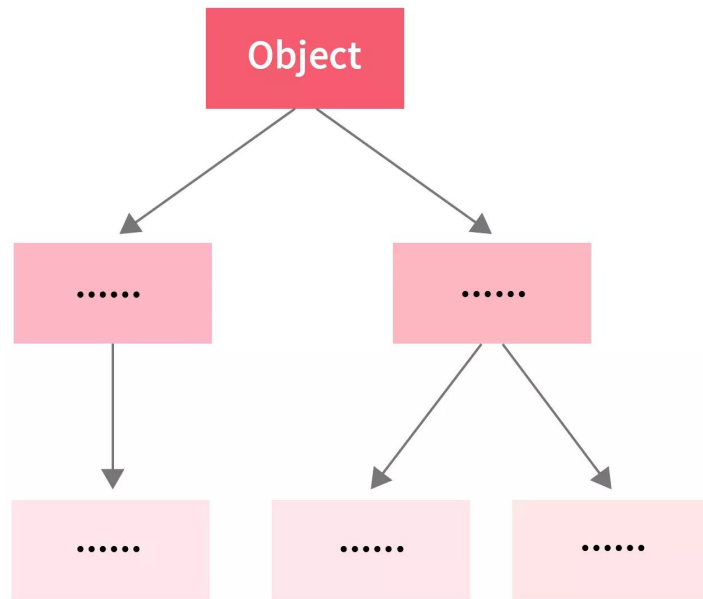
Conceito

- Quando uma classe usa a relação de herança, podemos dizer que essa classe possui um relacionamento chamado “**É um**” com a classe da qual ela herda.
- Tal relação também indica que **uma classe é do mesmo tipo que outra.** Assim, nos exemplos anteriores, podemos dizer que Gerente “é um” Funcionario

Classe Object - O topo da hierarquia de herança

Conceito

- Todas as classes em Java descendem de uma classe, chamada **Object**, mesmo que a declaração **extends Object** seja omitida, a classe Object é considerada a classe raiz da hierarquia de todas as classes Java, sendo, portanto, ancestral de todas as classes da linguagem.



Classe Object - O topo da hierarquia de herança

Conceito

- Métodos herdados
 - a classe Object provê múltiplos métodos que são **automaticamente herdados por todas as classes Java**

